

Manual de Usuario del Compilador HULK

David Sánchez

July 3, 2025

Contents

1	Introducción	2
2	Instalación y Compilación	2
2.1	Comandos del Makefile	2
2.2	Compilar y Ejecutar un Programa HULK	2
3	Sintaxis del Lenguaje HULK	2
3.1	Estructura General	2
3.2	Tipos de Datos	3
3.3	Gramática	3
4	Ejemplos de Código	3
4.1	Hola Mundo	3
4.2	Declaración de Variables y Expresiones	4
4.3	Definición de Funciones	4
4.4	Estructuras Condicionales	4

1 Introducción

Este documento es una guía de usuario para el compilador del lenguaje de programación HULK. El objetivo es proporcionar instrucciones claras sobre cómo escribir código en HULK, cómo compilarlo y cómo ejecutarlo.

2 Instalación y Compilación

Para utilizar el compilador, necesitarás tener Flex y Bison instalados, así como un compilador de C++ (como g++). El proyecto se compila utilizando un 'makefile'.

2.1 Comandos del Makefile

El 'makefile' proporcionado automatiza las tareas de compilación y limpieza. Los principales comandos son:

- **Limpiar el proyecto ('make clean'):** Elimina todos los ficheros objeto ('.o') y el ejecutable final. Es útil para realizar una compilación desde cero.

```
make clean
```

- **Compilar el proyecto ('make compile'):** Compila todo el código fuente y genera el ejecutable 'hulk' en el directorio 'hulk/'.

```
make compile
```

- **Ejecutar el compilador ('make execute'):** Este comando ejecuta el compilador. Si el ejecutable no existe o el código fuente ha cambiado, primero compilará el proyecto y luego lo ejecutará.

```
make execute
```

2.2 Compilar y Ejecutar un Programa HULK

Una vez que el compilador 'hulk' ha sido generado, puedes usarlo para compilar y ejecutar tus propios ficheros de código HULK (con extensión '.hulk').

Para compilar y ejecutar un fichero, usa el siguiente comando:

```
./hulk/hulk < ruta/a/tu/fichero.hulk
```

El compilador procesará el fichero y, si no hay errores, ejecutará el código.

3 Sintaxis del Lenguaje HULK

A continuación se presenta un resumen de la sintaxis del lenguaje HULK.

3.1 Estructura General

Un programa en HULK consiste en una secuencia de declaraciones y expresiones. Las expresiones deben terminar con un punto y coma (;).

```
// Ejemplo de un programa simple
let a = 42 in print(a);
```

3.2 Tipos de Datos

HULK soporta los siguientes tipos de datos básicos:

- **Number:** Números, tanto enteros como decimales.
- **String:** Cadenas de texto.
- **Boolean:** Valores 'true' o 'false'.

También soporta la definición de nuevos tipos (clases).

3.3 Gramática

A continuación se muestra la gramática del lenguaje en formato BNF:

```
input -> line
      | lines_block

lines_block -> { lines }

lines -> epsilon
      | non_empty_lines

non_empty_lines -> line
                | non_empty_lines line

line -> expr ;
      | func_asign ;
      | type_node_decl

expr -> arit_op
      | bool_expr
      | while_expr
      | string
      | id_expr
      | func_call
      | let_assign
      | if conditional
      | member_access_expr
      | expr := expr
      | expr = expr
      | expr @ expr
      | expr @@ expr

func_asign -> function ID ( args_list ) => expr
          | function ID ( args_list ) : ID => expr
          | function ID ( args_list ) { lines }
          | function ID ( args_list ) : ID { lines }

let_assign -> let var_assign_list in expr
           | let var_assign_list in lines_block

// ... (resto de la gramática)
```

4 Ejemplos de Código

4.1 Hola Mundo

```
print("Hello, World!");
```

4.2 Declaración de Variables y Expresiones

```
let message = "El resultado es: " in {  
  let a = 10;  
  let b = 32;  
  print(message @@ (a + b));  
};
```

4.3 Definición de Funciones

```
function add(a: Number, b: Number): Number => a + b;  
  
print(add(5, 7));
```

4.4 Estructuras Condicionales

```
let a = 10 in  
  if (a > 5) {  
    print("a es mayor que 5");  
  } else {  
    print("a no es mayor que 5");  
  }
```